AIOps Predictive Autoscaler

Command Reference — Setup, Debugging & Data Collection Log

This is a running reference of every command used while setting up the Kind + Online Boutique + Prometheus/Grafana testbed for the predictive-autoscaling project — from first install through the recurring debugging cycles (RBAC corruption, OOMKilled pods, DNS resolution failures) to load-testing and metric export. Commands are grouped by task, in the order they were actually needed, so this can be used as a step-by-step redo guide after any crash or fresh machine.

> ■ *Whenever the cluster shows CrashLoopBackOff, Unauthorized errors, or DNS lookup failures after a laptop sleep/reboot, jump to Section 6 (Recovery Playbook) first before rebuilding from scratch.*

1 One-Time Tool Installation

Run once per machine. Confirms Docker, kubectl, Kind, Helm and k6 are all present.

1.1 Docker

```
sudo apt remove docker docker-engine docker.io containerd runc -y
sudo apt install ca-certificates curl gnupg -y
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release &amp;amp;&amp;amp; echo
$VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y

# allow running docker without sudo
sudo usermod -aG docker $USER
# then open a FRESH terminal (or reboot) before continuing
```

1.2 kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

1.3 Kind

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
kind version
```

1.4 inotify limits (prevents 'Preparing nodes' failure)

```
sudo sysctl fs.inotify.max_user_watches=524288
sudo sysctl fs.inotify.max_user_instances=512
echo "fs.inotify.max_user_watches = 524288" | sudo tee -a /etc/sysctl.conf
echo "fs.inotify.max_user_instances = 512" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

1.5 Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

1.6 k6 (load testing) — direct binary, avoids GPG keyserver issues

```
curl -L
https://github.com/grafana/k6/releases/download/v0.53.0/k6-v0.53.0-linux-amd64.tar.gz -o
k6.tar.gz
tar -xzf k6.tar.gz
sudo mv k6-v0.53.0-linux-amd64/k6 /usr/local/bin/
k6 version
```

■ *The apt/GPG-keyserver method for k6 repeatedly failed here (NO_PUBKEY, unsupported-filetype warnings). The direct binary download above is the reliable path — use it first next time.*

2 Cluster & Application Setup

2.1 kind-config.yaml (2 workers — see Section 5 for why not 3)

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
```

2.2 Create cluster

```
mkdir -p ~/aiops-autoscaler &amp;&amp; cd ~/aiops-autoscaler
# save kind-config.yaml here (content above)
kind create cluster --name aiops-cluster --config kind-config.yaml
kubectl get nodes
```

2.3 Deploy Online Boutique (target application)

```
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-demo/main/release/kubernetes-manifests.yaml
kubectl get pods --watch # Ctrl+C once all Running
```

2.4 Verify frontend is reachable

```
kubectl port-forward svc/frontend 8080:80
# then open http://localhost:8080 in a browser
```

3 Monitoring Stack (Prometheus + Grafana)

3.1 Install (lightweight — Alertmanager off, lower memory requests)

```
helm install prometheus prometheus-community/kube-prometheus-stack \
--namespace monitoring --create-namespace \
--set alertmanager.enabled=false \
--set grafana.resources.requests.memory=128Mi \
--set prometheus.prometheusSpec.resources.requests.memory=256Mi
kubectl get pods -n monitoring
```

3.2 Port-forward Prometheus + Grafana

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80
```

3.3 Grafana login

```
# username is always: admin
kubectl get secret -n monitoring prometheus-grafana \
-o jsonpath="{.data.admin-password}" | base64 --decode
echo
```

Dashboard → **Kubernetes / Compute Resources / Namespace (Pods)** → set the **namespace** field to `default` to see live CPU/memory per service.

4 Load Generation & Metric Export

4.1 loadtest.js — simulated rush-hour traffic (k6)

Stages: 5 users (2m) → 50 users (1m) → 50 users (2m) → 5 users (1m) → 5 users (2m) → 80 users (1m) → 80 users (2m) → 5 users (1m). ~11 minutes total, two burst intensities.

```
cd ~/aiops-autoscaler
k6 run loadtest.js
```

4.2 export_metrics.py — pull Prometheus data to CSV

Requires the Prometheus port-forward (3.2) running in another terminal. Produces `cpu_metrics.csv` (per-pod) and `total_cpu_timeseries.csv` (aggregated — this is the one used for forecasting).

```
pip3 install requests pandas --break-system-packages
python3 export_metrics.py
```

5 Resource & Memory Fixes (OOMKilled)

Symptom: pods (usually `recommendationservice`, `emailservice`) stuck in `CrashLoopBackOff`, restart count climbing. Confirmed via Exit Code 137 = OOMKilled.

```
free -h # check available RAM
kubectl describe pod -l app=recommendationservice | grep -A 5 'Last State'
# look for: Reason: OOMKilled Exit Code: 137
```

Fixes applied, in order of effort:

```
# 1. Close background apps (browser tabs, unused programs)
# 2. Reduce worker node count 3 -> 2 in kind-config.yaml, then recreate cluster
# 3. Install Prometheus stack lightweight (see 3.1 flags)
```

6 Recovery Playbook (After Sleep / Reboot / Crash)

6.1 First — check if the existing cluster is still healthy (fastest path)

```
docker ps -a | grep aiops
# if containers show 'Exited', start them:
docker start aiops-cluster-control-plane aiops-cluster-worker aiops-cluster-worker2

kubectl get nodes
kubectl get pods
kubectl get pods -n kube-system # check coredns is 1/1, no CrashLoopBackOff
kubectl get pods -n monitoring
```

■ *If everything above shows Running / Ready with no CrashLoopBackOff or Unauthorized errors — stop here, just re-open the port-forwards (3.2) and continue. Do NOT rebuild.*

6.2 If CoreDNS / kube-proxy show CrashLoopBackOff or 0/1 Ready

Seen after repeated laptop sleep/wake cycles — Kind's iptables/networking state corrupts. Symptom in the app: 'dns: A record lookup error... connection refused'.

```
kubectl rollout restart deployment coredns -n kube-system
kubectl delete pod -n monitoring -l app.kubernetes.io/name=grafana
# wait 30-60s, then re-check kubectl get pods -n kube-system
```

If restart doesn't clear it, rebuild the cluster fresh:

```
kind delete cluster --name aiops-cluster
kind create cluster --name aiops-cluster --config kind-config.yaml
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-dem
o/main/release/kubernetes-manifests.yaml
helm install prometheus prometheus-community/kube-prometheus-stack \
--namespace monitoring --create-namespace \
--set alertmanager.enabled=false \
--set grafana.resources.requests.memory=128Mi \
--set prometheus.prometheusSpec.resources.requests.memory=256Mi
```

6.3 If Prometheus-operator logs show 'Unauthorized' (RBAC corrupted)

```
kubectl logs -n monitoring deployment/prometheus-kube-prometheus-operator --tail=50
# look for: 'Failed to watch ... Unauthorized'

# fix: full stack reinstall regenerates RBAC roles cleanly
helm uninstall prometheus -n monitoring
helm install prometheus prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--set alertmanager.enabled=false \
--set grafana.resources.requests.memory=128Mi \
--set prometheus.prometheusSpec.resources.requests.memory=256Mi
```

6.4 Grafana 'stale IP' after any Prometheus reinstall

Grafana's DNS cache holds the old Prometheus ClusterIP → i/o timeout errors in Explore. Fix:

```
kubectl rollout restart deployment coredns -n kube-system
kubectl delete pod -n monitoring -l app.kubernetes.io/name=grafana
# then restart the port-forward and hard-refresh the browser (Ctrl+Shift+R)
```

7 Git / GitHub Reference

7.1 First-time identity

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

7.2 Authentication — GitHub requires a Personal Access Token, not a password

```
# Generate at: https://github.com/settings/tokens -> Generate new token (classic)
# scope needed: repo
git config --global credential.helper store # remembers the token after first use
```

7.3 Large file rejected (>100MB)

```
git rm --cached path/to/large_file.pkl
echo "*.pkl" >> .gitignore
git add .gitignore
git commit --amend -m "Baseline implementation"
git push origin main
```

8 Quick Diagnostics Cheat-Sheet

```
kubectl get nodes # cluster up?
kubectl get pods # app healthy?
kubectl get pods -n monitoring # observability healthy?
kubectl get pods -n kube-system # coredns / kube-proxy healthy?
kubectl describe pod &lt;name&gt; | grep -A5 'Last State' # why did it restart?
kubectl logs deployment/&lt;name&gt; --tail=50 # app-level errors
free -h # RAM pressure?
docker ps -a | grep aiops # are cluster containers even running?
```